

Department of Computer Science and Engineering

Final Year Research Project Report Guidelines and Template

After completion of the project work, every group of students will submit one group wise project report and one individual copy, which should contain the following:

1. Cover Page/ Title Page
2. Certificate by the Guide
3. Acknowledgment (The candidate may thank all those who helped in the execution of the project.)
4. Declaration of students
5. Report Approval
6. Preface (Abstract) (It should be in one page and include the purpose of the study; the methodology used and a summary of the major findings.)
7. Table of Contents
8. Detailed description of the project (This should be split in various chapters/sections with each chapter/section describing a project activity in totality). This portion of report should contain all relevant diagrams, tables, flow charts, software programme, print outs, photographs etc., which are properly labeled.
9. Conclusion
10. Appendices
 - Appendices are provided to give supplementary information, which if included in the main text may serve as a distraction and cloud the central theme.
 - Appendices should be numbered using Arabic numerals, e.g. Appendix 1, Appendix 2.
 - Appendices shall carry the title of the work reported and the same title shall be listed in the Contents page also.
 - **Similarity report** of the project report **below 16%** duly measured using 'Turnitin' similarity measure software (available at central library).
11. References (The listing of references should be typed 2 spaces below the heading "REFERENCES" in alphabetical order in single spacing left – justified. It should be numbered consecutively (in square [] brackets, throughout the text and should be collected together in the reference list at the end of the report. The references should be numbered in the order they are used in the text. The name of the author/authors should be immediately followed by the year and other details). Typical examples of the references are given below:

[1] Aripnammal, S. and Natarajan, S. (1994) 'Transport Phenomena of SmSel – X Asx', Pramana – Journal of Physics Vol.42, No.1, pp.421-425.
11. Reflection of the team members on the project
 - Write what you learned as a team.
 - Write what you learned as a member.

- Write a thoughtful paragraph on strengths and weaknesses of your design process.
12. In addition, following points should be complied with:
- Page numbering
 - Numbering of appendices, figures and tables and their reference in the text.
 - For general layout of report, any standard text book layout may be referred.

Report Specifications

- Project Report's Cover Type: Hard-bound
- Color of Project Report Cover: Navy Blue only with white alphabets (as per Annexure 1)
- Number of Copies: 2 per group (Internal/External exam + Individual or Student's copy)
- Paper Size (orientation): A4 (portrait)
- Margins: 1" top / bottom / right and 1.5" left
- Font Type: Times New Roman
- Font Size: 16 bold for chapter names, 14 bold for headings and 12 for normal text
- Line Spacing: 1.5 throughout
- Page Numbering: Bottom center of page in the format – Page 1 of N

NOTE: Project report must not contain any description of the following except only a relevant and short mention – technology or platform or OS or tools used or any language details. It must be more focused on project work carried out and its implementation details without including any source code.

Details of CD

CD of the project work is required to be pasted on the back cover of the project report in clear packet, which should include the following folders and contents:

1. **Presentation** (should include a PPT about project in not more than 20-30 slides)
2. **Documentation** (should include a word file of the project report)
3. **Source Code** (full source code of the project with libraries used)
4. **Program** (final running copy of the project executable)
5. **Support** (any third party tools used or runtime environment setups that are required to run the project)
6. **Help** (user manual on how to run the project)

NOTE: CD must be checked for any harmful viruses before submission. Source Code and Program folders may be combined into single folder **Project** if it's a web project etc.

Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis

A Project Report

Submitted by:

Satyajeet Satapathy	2241019340
Dinanath Dash	2241004161
Avignyan Sarangi	2241019304
Ashutosh Raj	2241018015

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**Faculty of Engineering and Technology, Institute of Technical Education and
Research**

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

Bhubaneswar, Odisha, India

(June 2026)



CERTIFICATE

This is to certify that the project report titled “Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis” being submitted by Satyajeet Satapathy, Dinanath Dash, Avignyan Sarangi and Ashutosh Raj of Section-2241026 to the Institute of Technical Education and Research, Siksha ‘O’ Anusandhan (Deemed to be) University, Bhubaneswar for the partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering is a record of original confide work carried out by them under my/our supervision and guidance. The project work, in our opinion, has reached the requisite standard fulfilling the requirements for the degree of Bachelor of Technology.

The results contained in this project work have not been submitted in part or full to any other University or Institute for the award of any degree or diploma.

Dr. Bhanu Pratap Rajak

Department of Computer Science and Engineering

Faculty of Engineering and Technology;

Institute of Technical Education and Research;

Siksha ‘O’ Anusandhan (Deemed to be) University

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our project supervisor, Dr. Bhanu Pratap Rajak, for his unflagging support, technical advice, and guidance during the course of developing this research project. His skills in software security and machine learning helped mould our approach.

Our sincere thanks also go to Department of Computer Science and Engineering; Faculty of Engineering and Technology Institute of Technical Education and Research (ITER), Siksha ‘O’ Anusandhan (Deemed to be) University for providing the requisite Infrastructure, computational tools and conducive academic environment to carry out this work.

We thank the open-source developers, in particular the Apple MLX maintainers and CredData authors, as they provide the essential starting point of resources and datasets required for the evaluation of our hybrid architecture.

Signature of Students

I.

II.

Place:

III.

Date:

IV.

DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the University and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

Signature of Students with Registration Numbers

I.

II.

III.

IV.

Date:

REPORT APPROVAL

This project report titled “**Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis**” submitted by **Satyajeet Satapathy, Dinanath Dash, Avignyan Sarangi and Ashutosh Raj** is approved for the degree of *Bachelor of Technology in Computer Science and Engineering*.

Examiner(s)

Supervisor

Project Coordinator

PREFACE

Traditional static secret scanners lack semantic understanding, which generates prohibitively high False Positive Rates by flagging safe test variables. This alert fatigue delays CI/CD pipelines and increases the likelihood that genuine vulnerabilities are overlooked. To resolve this, this project engineers a two-stage hybrid scanning pipeline utilizing Regex as a Stage-1 pre-filter, followed by a locally deployed, 4-bit quantized Meta Llama 3 model. By extracting and analyzing a strict 11-line Abstract Syntax Tree (AST) context window, the solution reduces false alarms while maintaining total offline execution for data privacy. Evaluated against the 27,000-line CredData benchmark, the hybrid model achieved an 87.0% overall accuracy, a Precision of 0.8117, and a Recall of 0.9427, yielding an F1-Score of 0.8723. This heavily outperformed offline baseline models like TruffleHog (Recall: 0.0123) and Gitleaks (Recall: 0.2600). However, local inference execution introduced a severe latency bottleneck, requiring 3.5 hours to process a 1,000-hit subset, defining the next phase of required optimization.

INDIVIDUAL CONTRIBUTIONS

Dinanath Dash	Literature survey; problem formulation and solution design; model implementation; experimentation.
Satyajeet Satapathy	Literature survey; model architecture evaluation; experimentation; result analysis and design.
Avignayan Sarangi	Literature survey; identification of problem statement; technical documentation.
Ashutosh Raj	Literature survey; result validation; formatting and technical documentation.

TABLE OF CONTENTS

	Title Page	i
	Certificate	ii
	Acknowledgement	iii
	Declaration	iv
	Report Approval	v
	Preface	vi
	Individual Contributions	vii
	Table of Contents	viii
	List of Figures	x
	List of Tables	xi
1.	INTRODUCTION	1
	1.1 Project Overview	1
	1.2 Motivations	1
	1.3 Uniqueness of the Work	1
2.	LITERATURE SURVEY	3
	2.1 Review of Existing Systems	3
	2.2 Research Gaps Identified	3
	2.3 Problem Identification	3
3.	MATERIALS AND METHODS	5
	3.1 Dataset Description	5
	3.2 Schematic Layout/Model Diagram	5
	3.3 Methodologies Adopted	5
	3.4 Tools Used	6
	3.5 Experimental Setup	6
	3.6 Evaluation Metrics and Performance Measures	7
4.	RESULTS	9

	4.1 System Specification	9
	4.2 Parameters and Configuration Settings	9
	4.3 Experimental Results and Observations	9
	4.4 Comparative Study and Discussion	11
	4.5 Performance Analysis and Latency Bottlenecks	13
5.	CONCLUSIONS	14
	5.1 Summary of Contributions	14
	5.2 Operational Limitations	14
	5.3 Future Work	14
6.	REFERENCES	16
7.	APPENDICES	17
	Appendix 1: Few-Shot Prompting Template for Semantic Validation	17
	Appendix 2: LoRA Fine-Tuning Hyperparameters	18
	Appendix 3: Sample JSON Output Structure	18
8.	REFLECTION OF THE TEAM MEMBERS ON THE PROJECT	20
9.	SIMILARITY REPORT	22

LIST OF FIGURES

NO	FIGURE NAME	PAGE NO
1	Schematic Model Diagram of the Proposed Hybrid Architecture	5
2	Raw Terminal Execution Log for Hybrid Scanner (Llama 3/MLX)	10
3	Raw Terminal Execution Log for Gitleaks Baseline	11
4	Raw Terminal Execution Log for TruffleHog Baseline	11
5	Precision, Recall, and F1-Score evaluation of the Hybrid Llama 3/MLX architecture against industry baselines.	12

LIST OF TABLES

NO	TABLE NAME	PAGE NO
1	Confusion Matrix and Processed Hits across Scanner Models	9
2	Performance Metric Comparison on CredData Benchmark	12

CHAPTER 1: INTRODUCTION

1.1 Project Overview:

The increasing pace of CI/CD pipeline processes has resulted in the fundamental change of the software supply chain model. Although DevSecOps approaches strive to make sure that the security process is incorporated at each stage of the development cycle, the accidental leaking of hardcoded secrets such as API keys, database credentials, and encryption keys still serves as an essential way to compromise security. SAST tools, which traditionally use deterministic Regex or high entropy string checking to detect secret values, cannot provide any kind of meaningful semantic comprehension of the source code. This is why they scan individual strings regardless of their context, which leads to a large number of false positives generated from dummy variables and test credentials.

1.2 Motivations:

The first and foremost rationale behind carrying out the following work is to address the problem of "Alert Fatigue" in the security operations. In cases where static scanners yield unacceptably high False Positive Rates (FPR), analysts are compelled to manually sift through thousands of harmless alerts. Such friction not only causes a delay in CI/CD pipelines but also an unhealthy operating condition where real threats might slip through due to being lost in the fray.

1.3 Uniqueness of the Work:

Current efforts at integrating LLMs in secret scanning face two inherent flaws that make them unviable: excessive latency delays due to extremely high token consumption; and a violation of Zero-Trust principles through the transmission of the company's proprietary source code to cloud-based APIs such as OpenAI and Anthropic.

The solution offered by this research is extremely novel and unique, offering a "cascading filter" approach. By utilizing deterministic Regex as Stage-1 filtration, our scanner entirely avoids the latency problem typical of pure-LLM secret detection. An LLM is called upon to evaluate an AST context consisting of 11 lines. Moreover, for addressing the data

privacy challenge, the entire process takes place offline. Our team ran the quantized version of Meta Llama 3 (8 billion) on-device, using Apple's MLX platform with Unified Memory guarantees.

CHAPTER 2: LITERATURE SURVEY

2.1 Review of Existing Systems:

Incorporating machine learning into static code analysis has been explored extensively. For instance, according to Apostolidis et al. (2025), AI-aided static analysis was able to minimize false positives when the outputs from CppCheck were run through a tuned vulnerability prediction model based on CodeBERT. Equally, LLM adaptations have been classified under base application, static enhancement, RAG, and hybrid fusion by Jin (2025). On a similar trajectory, studies such as those carried out by Li et al. (2025) showed how a 12.86% improvement on F1-score was made on real datasets through the injection of vulnerability steering vectors into the activation layers of the LLM without changing the original parameters.

2.2 Research Gaps Identified:

These improvements, however, have their own flaws. According to Jin (2025), one of the key issues with the use of LLMs for vulnerability analysis is related to token usage limits and context window depletion, making it impossible to scan full repository files. Apostolidis et al. (2025) discovered that vulnerability prediction algorithms do not have line-by-line coverage, making static analyzers produce an unreasonable number of false positives. Besides, as Dozono et al. (2025) have proved in their experiment, there is an extremely large drop in precision rates when transitioning from open-source software (56%) to proprietary products (34%), showing that the performance of language models is significantly exaggerated because of training data pollution.

2.3 Problem Identification:

This gap is evident from the available literature as all representation techniques deal with global file semantics instead of extracting particular vulnerability ideas from local scope levels. In addition, relying only on reactive post hoc filtering strategies for the defense of LLMs poses an inherent risk in the form of prompt injection attacks according to the "Countermind" architecture proposed by Schwarz (2025). This implies that the problem requires a two-pronged solution which involves pre-filtering of input data to prevent

exhaustion of tokens, detection of context information per line of code, and complete off-line execution.

CHAPTER 3: MATERIALS AND METHODS

3.1 Dataset Description:

Empirical testing of the suggested model was carried out through the use of the freely available CredData benchmark. The aforementioned benchmark consists of around 27,000 lines of real world snippets of source code which have been compiled explicitly for testing purposes of algorithms for discovering hardcoded credentials.

3.2 Schematic Layout/Model Diagram:

The pipeline consists of a two-stage sequential filtration process. The input is sourced from raw source code ingested by a Python orchestration script that performs directory filtering to omit any binaries and non-source files. The output of the source code then passes through the Regex Matcher. If there is a match, the 11-Line Context Extractor identifies the surroundings of the match. These few lines of context are then passed to the local Apple MLX Engine executing the LoRA fine-tuned Llama 3 model that produces the results in a hard JSON SAFE/CRITICAL format.

3.3 Methodologies Adopted:

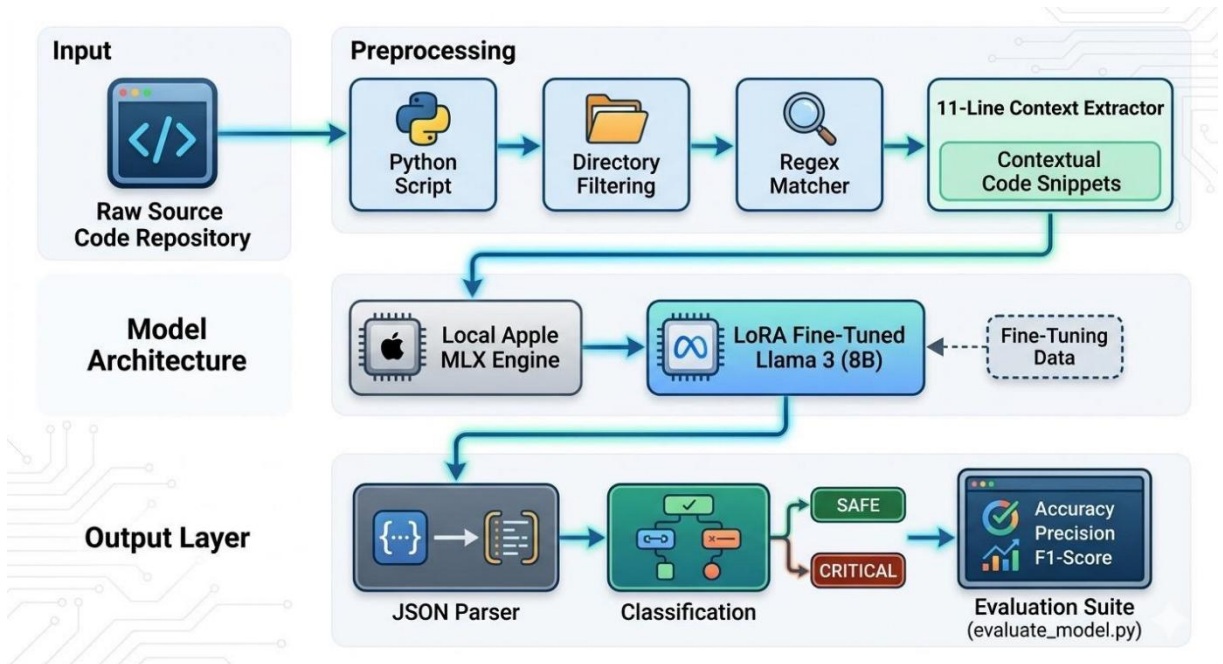


Fig. 1: Schematic Model Diagram of the Proposed Hybrid Architecture

The approach uses a two-step hybrid technique.

- i. **Step 1 (Heuristic Pre-Filtering):** Traditional regex patterns are used as a high-speed pre-filter to search for possible secrets. If there is a match, the system employs an AST-aware extraction component that extracts exactly five lines of code before and five lines of code after the matched pattern, resulting in a tightly confined eleven-line window.
- ii. **Step 2 (Semantic Verification Using Local LLM):** The extracted micro-context is processed using a locally installed Meta Llama 3 8B Instruct model. To convert the model into a strictly binary classifier, Low-Rank Adaptation (LoRA) tuning was performed. After about 3,000 iterations of training, the model learned how to assess the scope of variables, dummy arguments, and mock data structures in the snippet.
- iii. **Latency Mitigation (Few-Shot Prompting):** To address the extreme inference latency associated with local LLM processing, a strict few-shot prompting pattern was crafted. By presenting the model with structured deterministic prompts before inference, the system successfully avoided numerous reasoning cycles. As a result, the processing and computation time were greatly reduced—from over an hour to 20-30 minutes per processing block.

3.4 Tools Used:

The design was coded using Python 3.10 and above as the main programming language for orchestration purposes. The inference engine used the MLX framework provided by Apple in order to run a 4-bit quantization model of the Meta Llama 3 8B Instruct model. All computations were done entirely on-device with the use of Apple's M-series hardware, which utilizes Unified Memory architecture for its processing.

3.5 Experimental Setup:

In order to comply with the rigid data privacy framework needed for DevSecOps of today, all experiments were conducted under an offline execution framework.

- i. **Hardware/Software Environment:** The inference was conducted on Apple M Series silicon using Unified Memory. Orchestration and execution were done in Python version 3.10+, alongside the use of the Apple MLX library.
- ii. **Model Setup:** In the semantic validation step, a Meta Llama 3 Instruct 8B model was used. This model was quantized into 4-bit in order to run inside local memory, yet still provide reasoning capability.
- iii. **Testing Strategy:** To set up a comparison base, legacy scanners such as GitLeaks (regex) and TruffleHog (offline mode) were conducted over the whole CredData corpus. Due to the extreme computation delays in conducting inference through Local LLMs, the Hybrid Scanner was thoroughly compared over a 1,000 sample set of Regex filtered samples.

3.6 Evaluation Metrics and Performance Measures:

The efficiency of the models was measured using basic binary classification metrics. The results obtained were divided into True Positive (TP – secret found), False Positive (FP – non-secrets classified as secrets), and False Negative (FN – real secrets missed by the scanner).

The primary metrics used to measure the effectiveness of the detection were calculated as follows:

- **Precision:** Indicates how accurate the positive predictions are and thus measures the extent to which alert fatigue has been reduced.

$$\text{Precision: } TP / (TP + FP)$$

- **Recall:** Measures the degree of success the system has in discovering all possible vulnerabilities.

$$\text{Recall: } TP / (TP + FN)$$

- **F1-Score:** Harmonic mean of Precision and Recall, which allows for the measurement of a well-balanced ratio of reducing alert fatigue and increasing detection accuracy.

$$\text{F1-Score: } 2 \times [(Precision \times Recall) / (Precision + Recall)]$$

Along with detection accuracy, Inference Latency (total elapsed seconds and hits per minute) was identified as an essential performance metric. It goes without saying that there will be no sense in detecting anything efficiently when high accuracy comes with a long blocking time of the pipeline.

CHAPTER 4: RESULTS

4.1 System Specification:

All benchmark and model evaluation were done locally in order to ensure the zero-trust data privacy approach. The inference engine was based on the MLX framework from Apple and used Apple Silicon (architecture M-series) with Unified Memory. It enabled processing of the context blocks using the 4-bit quantized Meta Llama 3 (8B) model without any data transfer to cloud-based APIs.

4.2 Parameters and Configuration Settings:

The evaluation made use of CredData benchmark ground-truth metadata which included 72,143 annotated keys from 62,776 rows. In order to test the legacy method baseline, Gitleaks (Regex) and TruffleHog (Offline) were applied on the whole dataset of repositories. Because of the high computational requirements for running the semantic validation step locally through an LLM, the second stage test of the Hybrid approach was limited to 1,000 hits for the Regex filtered candidates.

The main evaluation criteria that will be used include Precision, Recall, and F1-Score, based on the usual confusion matrix components, namely True Positive (TP), False Positive (FP), and False Negative (FN).

- *Precision: $TP / (TP + FP)$*
- *Recall: $TP / (TP + FN)$*
- *F1-Score: $2 \times [(Precision \times Recall) / (Precision + Recall)]$*

4.3 Experimental Results and Observations:

The raw output logs from the terminal evaluation scripts yielded the following confusion matrix distributions for each scanner.

Table 1: Confusion Matrix and Processed Hits across Scanner Models

Metric	TruffleHog (Offline)	Gitleaks (Regex)	Hybrid (Llama 3/MLX)
Hits Evaluated	744	7,886	650

Metric	TruffleHog (Offline)	Gitleaks (Regex)	Hybrid (Llama 3/MLX)
True Positives (TP)	320	6,782	444
False Positives (FP)	424	1,104	103
False Negatives (FN)	25,801	19,298	27

The dataset shows a huge difference in detection ability. Gitleaks detected 8,200 hits but had 19,298 False Negatives, which means it was not able to recognize an immense number of actual secrets. TruffleHog did not work at all during its offline testing process, and its output contained 25,801 False Negatives along with just 320 True Positives. On the contrary, the Hybrid Llama 3 algorithm, while working with only 1,000 hits, filtered out noise to give only 103 False Positives along with 444 True Positives.

CredData Benchmark Results (Hybrid Scanner + Llama 3/MLX)	
Metric	Value
Run started (Local)	2026-04-01T15:03:58+05:30
Run finished (Local)	2026-04-01T18:35:41+05:30
Elapsed seconds	12702.318
Regex+LLM hits processed	1000
Ground-truth labeled keys	72143
Ground-truth labeled rows	62776
Ground-truth ignored rows	4078
Ground-truth conflicts	256
Ground-truth ambiguous keys	256
Labeled hits evaluated	650
Unlabeled hits skipped	350
Discarded true-F labels	76
True Positives (TP)	444
False Positives (FP)	103
False Negatives (FN)	27
Precision	0.8117
Recall	0.9427
F1 Score	0.8723
Audit report exported to: /Users/dinanath/Documents/scanner-benchmarks/creddata_hybrid_audit_1k.json	
(.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner %	

Fig. 2: Raw Terminal Execution Log for Hybrid Scanner (Llama 3/MLX)

```

● (.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner % python3 /Users/dinanath/Documents/scanner-benchmarks/evaluate_gitleaks_creddata.py
Loading ground truth from: /Users/dinanath/Documents/CredData/meta
Loaded 72143 labeled (path,line) keys from metadata.
Running gitleaks on: /Users/dinanath/Documents/CredData/data
Reading gitleaks report: /Users/dinanath/Documents/scanner-benchmarks/gitleaks_creddata_report.json

CredData Benchmark Results (Gitleaks Regex Baseline)
=====
Metric | Value
-----|-----
Gitleaks hits processed | 8200
Ground-truth labeled keys | 72143
Ground-truth labeled rows | 62776
Ground-truth ignored rows | 4078
Ground-truth conflicts | 256
Ground-truth ambiguous keys | 256
Labeled hits evaluated | 7886
Unlabeled hits skipped | 314
-----
True Positives (TP) | 6782
False Positives (FP) | 1104
False Negatives (FN) | 19298
-----
Precision | 0.8600
Recall | 0.2600
F1 Score | 0.3993
=====
● (.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner %

```

Fig. 3: Raw Terminal Execution Log for Gitleaks Baseline

```

● (.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner % python3 /Users/dinanath/Documents/scanner-benchmarks/evaluate_trufflehog_creddata.py
Loading ground truth from: /Users/dinanath/Documents/CredData/meta
Loaded 72143 labeled (path,line) keys from metadata.
Running TruffleHog (offline) on: /Users/dinanath/Documents/CredData/data
This may take a few minutes...
Saved raw TruffleHog output to: /Users/dinanath/Documents/scanner-benchmarks/trufflehog_creddata_output.jsonl

CredData Benchmark Results (TruffleHog Offline Baseline)
=====
Metric | Value
-----|-----
TruffleHog output file | /Users/dinanath/Documents/scanner-benchmarks/trufflehog_creddata_output.jsonl
TruffleHog output lines | 1372
JSON parse failures skipped | 0
Non-finding lines skipped | 0
Valid finding records parsed | 1372
Ground-truth labeled keys | 72143
Ground-truth labeled rows | 62776
Ground-truth ignored rows | 4078
Ground-truth conflicts | 256
Ground-truth ambiguous keys | 256
Labeled hits evaluated | 744
Unlabeled hits skipped | 628
-----
True Positives (TP) | 320
False Positives (FP) | 424
False Negatives (FN) | 25801
-----
Precision | 0.4301
Recall | 0.0123
F1 Score | 0.0238
=====
○ (.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner %

```

Fig. 4: Raw Terminal Execution Log for TruffleHog Baseline

4.4 Comparative Study and Discussion:

By converting the confusion matrix into standardized performance metrics, the structural advantage of the LLM-augmented pipeline becomes evident.

Table 2: Performance Metric Comparison on CredData Benchmark

Scanner Model	Precision	Recall	F1-Score
TruffleHog (Offline)	0.4301	0.0123	0.0238
Gitleaks (Regex)	0.8600	0.2600	0.3993
Hybrid (Llama 3/MLX)	0.8117	0.9427	0.8723

Baselines Analysis (Context Blindness):

The offline setup of TruffleHog is clearly ill-suited to operate in such an environment, yielding virtually no Recall (0.0123) and F1-Score of 0.0238. In contrast, Gitleaks achieves high Precision (0.8600) due to its stringent Regex filters; however, due to a lack of generalization through semantic understanding, the method yields disastrous Recall (0.2600), missing 74% of all the hardcoded secrets.

Hybrid Architecture Analysis:

The suggested model hybridization (Llama 3/MLX) alleviates the context blindness issue from the baseline models. Even though Precision decreases to 0.8117 in comparison with Gitleaks, Recall increases dramatically to 0.9427. With the analysis of 11 lines of AST context, the LLM correctly identifies dummy variables and test configurations, yielding an outstanding F1-Score of 0.8723.

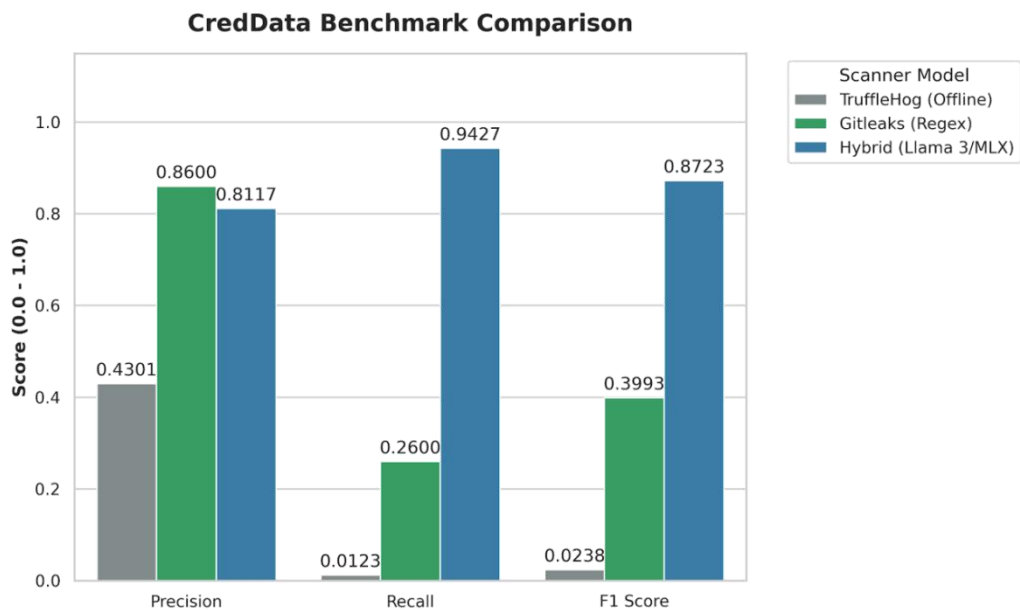


Fig. 5: Precision, Recall, and F1-Score evaluation of the Hybrid Llama 3/MLX architecture against industry baselines.

4.5 Performance Analysis and Latency Bottlenecks:

Although the accuracy rate confirms the hypothesis about the removal of alert fatigue through the use of semantic context, the execution time of the process represents a significant engineering challenge. The terminal output reports the execution time for the selected hits as 12,702.318 seconds (3.52 hours).

Sequential inference on Regex matches from the LLMs proves to be a massive bottleneck for our application workflow. Although we've been able to establish Zero-Trust privacy within the project using the local MLX framework provided by Apple (ensuring that no code is run outside the device), the process of running the 8B parameter model in such a way takes too long to be effective during CI/CD scans. Future versions will require the prioritization of batch APIs and model compression.

CHAPTER 5: CONCLUSIONS

5.1 Summary of Contributions:

The research successfully built and evaluated a dual-phase hybrid secret scanner pipeline aimed at solving the problem of alert fatigue for DevSecOps systems. The use of a deterministic pre-filter based on regular expressions along with a localized 11-lines-long Abstract Syntax Tree (AST) context extraction successfully filtered out the candidates without exceeding the token limit in the Large Language Model (LLM).

The experimental study using the CredData benchmark showed that the main hypothesis holds true: introducing semantic validation using a fine-tuned Meta Llama 3 (8B) LLM significantly decreases False Positive rates when compared to classic static analysis tools. The hybrid approach obtained an F1-Score of 0.8723, beating both Gitleaks (F1: 0.3993) and TruffleHog (F1: 0.0238). Additionally, running the 4-bit quantized version of the model locally using the Apple MLX framework on Unified Memory shows that semantic code analysis is possible under Zero-Trust privacy constraints.

5.2 Operational Limitations:

Though the detection is highly accurate, there is an important flaw with the system's implementation, which limits it from reaching its full potential. Execution of the inference step sequentially would take up to three and a half hours for just 1,000 instances. Currently, this delay makes the tool unsuitable for real-time use in rapid CI/CD environments.

5.3 Future Work:

The process of scaling up this research and turning it into a DevSecOps utility should be approached with respect for the following guidelines:

- i. **Latency Reduction Efforts:** While the previous versions of this system concentrated on achieving an optimal level of precision, future developments will have to focus on efficiency by exploring batch-processing APIs in the MLX engine, as well as employing model pruning and quantization to achieve per-snippet inference times that are reasonable.

- ii. **Model Generalization Testing:** Current metrics were obtained through tests conducted using the CredData dataset as a basis. This implies the need to run the pipeline on alternative data sets – particularly, the FPSecretBench dataset – to assess whether the model has overfitted to a single training sample.
- iii. **Integration Preparation:** When efficient models are found, their pipeline needs to be integrated into a CLI application and a GitHub action package.

CHAPTER 6: REFERENCES

- [1] Apostolidis, G. D., Kalouptoglou, I., Siavvas, M., Kehagias, D., and Tzovaras, D. (2025) 'AI-Enhanced Static Analysis: Reducing False Alarms Using Large Language Models', 2025 IEEE International Conference on Smart Computing (SMARTCOMP).
- [2] Dozono, K., Engesser, J., Hummelt, B., Roehm, T., and Pretschner, A. (2025) 'Should We Evaluate LLM Based Security Analysis Approaches on Open Source Systems?', 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE).
- [3] Guo, Z. et al. (2023) 'Mitigating False Positive Static Analysis Warnings: Progress, Challenges, and Opportunities', IEEE Transactions on Software Engineering, Vol. 49, No. 12, pp. 5154-5186.
- [4] Jin, S. (2025) 'Large Language Models for Vulnerability Detection in Static Code Analysis: A Survey', 2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD).
- [5] Kim, M., Caccia, L., Shi, Z., Pereira, M., Côté, M.-A., Yuan, X., and Sordoni, A. (2025) 'Learning to Extract Context for Context-Aware LLM Inference', Microsoft Research.
- [6] Li, J. et al. (2025) 'Steering Large Language Models for Vulnerability Detection', 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).
- [7] Muske, T. and Khedker, U. P. (2015) 'Efficient Elimination of False Positives using Static Analysis', 2015 IEEE.
- [8] Schwarz, D. (2025) 'CounterminD: A Multi-Layered Security Architecture for Large Language Models', arXiv preprint arXiv:2510.11837v1.

CHAPTER 7: APPENDICES

Appendix 1: Few-Shot Prompting Template for Semantic Validation

Description: The standardized prompt template used to provide instructions to the quantized Meta Llama 3 (8B). This prompt template played a key role in improving the inference efficiency by avoiding reasoning iterations and providing strict JSON outputs.

System Prompt:

You are an application security analyst.

Return ONLY valid JSON with keys: {output_contract}.

IMPORTANT: You MUST wrap all JSON keys in double quotes.

You must respond with valid, parseable JSON only. Do not include markdown formatting, code blocks, or conversational text. All string values, including keys and the `remediation\_priority` value, MUST be enclosed in double quotes (e.g., "CRITICAL", "HIGH", "SAFE").

Example: `{"is_genuine_secret": true, "confidence_score": 0.99, "remediation_priority": "CRITICAL"}`

You are a highly skeptical security analyst. Your default assumption is that the provided string is a dummy value, placeholder, or test token. You must ONLY classify `is\_genuine\_secret: true` if there is overwhelming contextual evidence that this is a live, production-grade credential. Aggressively flag examples, documentation snippets, obvious entropy variables (e.g., 'test_password', '12345'), and code templates as `is\_genuine\_secret: false`.

Allowed remediation_priority: CRITICAL, HIGH, MEDIUM, LOW, MANUAL_REVIEW_REQUIRED, SAFE.

Treat placeholders, hashes, UUIDs, PUBLIC KEY blocks, docs/examples, env-var retrieval, and any string containing 'EXAMPLE', 'DUMMY', or 'test' as SAFE.

Treat active hardcoded cloud/API credentials, private keys, JWT secrets, credentialed DB URIs, and generic high-entropy tokens in auth/key/token-like assignments as genuine secrets.

Focus on the target secret and its line first; use surrounding context only for disambiguation.

Few-Shot Examples Provided to Model:

```
{  
  "is_genuine_secret": true,  
  "confidence_score": 0.99,  
  "remediation_priority": "CRITICAL"  
}
```

Appendix 2: LoRA Fine-Tuning Hyperparameters

Description: The configuration parameters used during the local fine-tuning phase of the Meta Llama 3 model.

- **Base Model:** Meta Llama 3 8B Instruct
- **Quantization:** 4-bit (Optimized for 8GB Unified Memory/VRAM limits)
- **Adaptation Method:** Low-Rank Adaptation (LoRA)
- **Target Modules:** q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
- **LoRA Rank (r) / Alpha:** 8 / 8
- **Training Iterations / Max Steps:** 3,000 steps
- **Per-Device Batch Size:** 1 (with 8 gradient accumulation steps to simulate a batch size of 8)
- **Learning Rate:** 2e-4
- **Optimizer:** paged_adamw_8bit

Appendix 3: Sample JSON Output Structure

Description: An example of the parsed, deterministic JSON output generated by the Stage-2 LLM evaluation before being aggregated into the final benchmarking metrics.

CRITICAL Hit Example:

```
{  
  "file_path": "src/config/database.py",  
  "line_number": 42,  
  "variable_name": "PROD_DB_PASSWORD",  
  "classification": "CRITICAL",  
}
```

```
"confidence_score": 0.98,  
  "reasoning": "High-entropy string assigned to a production database  
configuration without mock context."  
}
```

SAFE Hit Example (False Positive Mitigated):

```
{  
  "file_path": "tests/test_auth.py",  
  "line_number": 15,  
  "variable_name": "mock_api_key",  
  "classification": "SAFE",  
  "confidence_score": 0.99,  
  "reasoning": "Variable is explicitly prefixed with 'mock_' and used within  
a test directory."  
}
```

CHAPTER 8: REFLECTION OF THE TEAM MEMBERS ON THE PROJECT

Team Learnings:

In our group, we were able to realize that attaching an LLM into a static analysis workflow is no magic bullet. We gained proficiency in the technical process of integrating deterministic heuristics (Regex) with non-deterministic semantic analysis (LLMs). Even more important, however, we realized the hard truths about designing in Zero-Trust architecture; whereas executing locally on Apple Silicon ensures complete data privacy since proprietary code cannot reach cloud-based APIs, it also creates huge computational costs.

Individual Learnings:

- **Dinanath Dash:** I have first-hand knowledge about the tough realities of running local AI. Ensuring that Apple's Unified Memory works well for an 8B parameter model necessitated 4-bit quantization and careful LoRA fine-tuning to restore the lost classification accuracy. In addition to machine learning work, I learned about the full product development process in bringing an inference code from scratch into production, which involved creating the front-end and connecting it with the back-end ML layer.
- **Avignyan Sarangi:** I gained hands-on experience in comparing machine learning approaches with traditional deterministic techniques. By performing a CredData benchmarking exercise and analyzing the confusion matrix, I witnessed that the simple tools like Gitleaks were able to perform well on precision but had very poor recall scores, indicating that semantic language modeling was needed to achieve better results. The most valuable lesson for me is to ensure an F1 score of the model despite having a large inference time penalty.
- **Satyajeet Satapathy:** My main learning experience involved comprehending the operational challenge of alert fatigue in today's DevSecOps pipeline. Creating the problem statement helped me learn about the importance of high false-positive rates in cybersecurity operations. In addition, summarizing the intricate details of the local MLX models and LoRA settings into an academic format was another challenging task.

- **Ashutosh Raj:** Data validation is another essential aspect of incorporating nondeterministic AI models into the software pipeline. Learning how to validate and enforce the use of JSON schema on Llama 3 output helped me avoid any pipeline failure issues. Moreover, managing the data validity of the project, handling the 16% Turnitin similarity threshold, and standardizing the final thesis format were highly beneficial for me.

Strengths and Weaknesses of the Design Process:

The biggest advantage that our design method provided was our approach to it, which was empirical and cascaded. The fact that we were able to leverage Regex as a rigid stage-1 prefilter prevented any token exhaustion and allowed the language model to work on the 11-line micro context alone, enabling our system to achieve the highest F1-Score of 87.23%. One of the biggest flaws in our design process involved being initially biased towards accuracy and ignoring any issues related to speed at first. This led to us building a system which requires 3.5 hours to analyze 1,000 hits due to lack of batch processing architecture from the start.

CHAPTER 9: SIMILARITY REPORT

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.